

Misinformation Models — Sprint 3: Multi-Task Learning

Driver notebook: runs all Sprint 2 models as baselines, adds LogReg MTL (cascaded POS). Train/dev only — test set not touched. Had help from Claude on implementation.

Import configuration and packages

```
In [1]: import sys
        from pathlib import Path
        sys.path.insert(0, str(Path.cwd()))

        import pandas as pd
        import numpy as np
        import torch                # CNN only
        import cnn_baseline as cnn  # CNN only

        # Running this will import FastText vector file, stored on HuggingFace and i
        from config import DATA_DIR, FASTTEXT_PATH, TARGETS

        from preprocess import preprocess # CNN only
        from metrics import compute_metrics, print_confusion_matrix, print_sklearn_r

        DEVICE = torch.device(                # CNN only
            "mps" if torch.backends.mps.is_available() else
            "cuda" if torch.cuda.is_available()         else
            "cpu"
        )
        print(f"Device: {DEVICE}")             # CNN only
        print(f"Targets: {TARGETS}")
```

```
/Users/jennifer/miniforge3/envs/colx_misinformation/lib/python3.12/site-pack
ages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter
r and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_insta
ll.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set
a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
Device: mps
```

```
Targets: ['opinion_label', 'misinformation_label']
```

```
In [2]: # Load data (shared across all models)
        train_rows = cnn.load_csv(DATA_DIR / "mis_df_train.csv")
        dev_rows   = cnn.load_csv(DATA_DIR / "mis_df_dev.csv")
        print(f"Train: {len(train_rows)} | Dev: {len(dev_rows)}")
```

```
Train: 600 | Dev: 200
```

Run models

```
In [ ]: # Each model should produce (preds, labels, probs) and store them in `results`
# Add new models below following the same pattern.
results = {}

#added metrics_cache for later use in ordering all models by Macro F1 and macro
metrics_cache = {}
```

CNN Baseline

```
In [ ]: #CNN Baseline
vocab = cnn.build_vocab(train_rows, preprocess)
embed_matrix = cnn.load_fasttext_vectors(FASTTEXT_PATH, vocab)
train_loader = cnn.make_loader(train_rows, vocab, shuffle=True, tokenize_fr
dev_loader = cnn.make_loader(dev_rows, vocab, shuffle=False, tokenize_fr
for target in TARGETS:
    model = cnn.TextCNN(len(vocab), embed_matrix).to(DEVICE)
    model = cnn.train_model(model, train_loader, dev_loader, train_rows, DEV
                        train_targets=[target])
    key = f"TextCNN - {target}"
    results[key] = cnn.predict(model, dev_loader, DEVICE, target=target)

#save metrics for use in final table
metrics_cache[key] = compute_metrics(*results[key])
```

Loading FastText vectors from /Users/jennifer/.cache/huggingface/hub/datasets--COLX523--fasttext-cc-en-300/snapshots/63ba06b23eb770a6fe0f6c17b0de981e43aec6aa/cc.en.300.vec ...

3875/4091 vocab tokens found in FastText vectors (94.7%)

Epoch	1		loss=0.7467		avg_dev_f1=0.6976	(opinion_label: 0.6976)
Epoch	2		loss=0.6497		avg_dev_f1=0.6928	(opinion_label: 0.6928)
Epoch	3		loss=0.6010		avg_dev_f1=0.7093	(opinion_label: 0.7093)
Epoch	4		loss=0.5371		avg_dev_f1=0.7014	(opinion_label: 0.7014)
Epoch	5		loss=0.5012		avg_dev_f1=0.7177	(opinion_label: 0.7177)
Epoch	6		loss=0.4437		avg_dev_f1=0.7172	(opinion_label: 0.7172)
Epoch	7		loss=0.3909		avg_dev_f1=0.6900	(opinion_label: 0.6900)
Epoch	8		loss=0.3446		avg_dev_f1=0.7118	(opinion_label: 0.7118)
Epoch	9		loss=0.2789		avg_dev_f1=0.7261	(opinion_label: 0.7261)
Epoch	10		loss=0.2521		avg_dev_f1=0.7159	(opinion_label: 0.7159)
Epoch	11		loss=0.2244		avg_dev_f1=0.7081	(opinion_label: 0.7081)
Epoch	12		loss=0.1804		avg_dev_f1=0.7081	(opinion_label: 0.7081)
Epoch	13		loss=0.1683		avg_dev_f1=0.7166	(opinion_label: 0.7166)
Epoch	14		loss=0.1379		avg_dev_f1=0.7144	(opinion_label: 0.7144)

Early stopping (no improvement for 5 epochs). Best F1: 0.7261

Epoch	1		loss=1.0064		avg_dev_f1=0.6711	(misinformation_label: 0.6711)
Epoch	2		loss=0.8647		avg_dev_f1=0.7476	(misinformation_label: 0.7476)
Epoch	3		loss=0.7113		avg_dev_f1=0.7921	(misinformation_label: 0.7921)
Epoch	4		loss=0.5219		avg_dev_f1=0.8160	(misinformation_label: 0.8160)
Epoch	5		loss=0.3933		avg_dev_f1=0.8492	(misinformation_label: 0.8492)
Epoch	6		loss=0.2972		avg_dev_f1=0.8588	(misinformation_label: 0.8588)
Epoch	7		loss=0.2361		avg_dev_f1=0.8773	(misinformation_label: 0.8773)
Epoch	8		loss=0.1795		avg_dev_f1=0.8684	(misinformation_label: 0.8684)
Epoch	9		loss=0.1390		avg_dev_f1=0.8684	(misinformation_label: 0.8684)
Epoch	10		loss=0.1114		avg_dev_f1=0.8456	(misinformation_label: 0.8456)
Epoch	11		loss=0.0886		avg_dev_f1=0.8609	(misinformation_label: 0.8609)
Epoch	12		loss=0.0691		avg_dev_f1=0.8684	(misinformation_label: 0.8684)

Early stopping (no improvement for 5 epochs). Best F1: 0.8773

CNN Transfer Learning

```
In [ ]: # CNN Transfer Learning
cnn_model = cnn.TextCNN(len(vocab), embed_matrix).to(DEVICE)
cnn_model = cnn.train_model(cnn_model, train_loader, dev_loader, train_rows,
for target in TARGETS:
    key = f"TextCNN Transfer - {target}"
    results[key] = cnn.predict(cnn_model, dev_loader, DEVICE, target=target)
    #save metrics for use in final table
    metrics_cache[key] = compute_metrics(*results[key])
```

```

Epoch 1 | loss=1.7759 | avg_dev_f1=0.6759 (opinion_label: 0.6837 | misinfo
rmation_label: 0.6682)
Epoch 2 | loss=1.5748 | avg_dev_f1=0.7325 (opinion_label: 0.6933 | misinfo
rmation_label: 0.7717)
Epoch 3 | loss=1.3835 | avg_dev_f1=0.7179 (opinion_label: 0.6726 | misinfo
rmation_label: 0.7631)
Epoch 4 | loss=1.2139 | avg_dev_f1=0.7420 (opinion_label: 0.6869 | misinfo
rmation_label: 0.7971)
Epoch 5 | loss=1.0466 | avg_dev_f1=0.7503 (opinion_label: 0.6846 | misinfo
rmation_label: 0.8160)
Epoch 6 | loss=0.9388 | avg_dev_f1=0.7675 (opinion_label: 0.6893 | misinfo
rmation_label: 0.8456)
Epoch 7 | loss=0.8396 | avg_dev_f1=0.7682 (opinion_label: 0.6869 | misinfo
rmation_label: 0.8496)
Epoch 8 | loss=0.7500 | avg_dev_f1=0.7754 (opinion_label: 0.6976 | misinfo
rmation_label: 0.8533)
Epoch 9 | loss=0.6581 | avg_dev_f1=0.7772 (opinion_label: 0.7029 | misinfo
rmation_label: 0.8515)
Epoch 10 | loss=0.5871 | avg_dev_f1=0.7609 (opinion_label: 0.6782 | misinfo
rmation_label: 0.8435)
Epoch 11 | loss=0.5143 | avg_dev_f1=0.7852 (opinion_label: 0.6963 | misinfo
rmation_label: 0.8742)
Epoch 12 | loss=0.4496 | avg_dev_f1=0.7812 (opinion_label: 0.6940 | misinfo
rmation_label: 0.8684)
Epoch 13 | loss=0.4127 | avg_dev_f1=0.7709 (opinion_label: 0.6884 | misinfo
rmation_label: 0.8533)
Epoch 14 | loss=0.3497 | avg_dev_f1=0.7770 (opinion_label: 0.6931 | misinfo
rmation_label: 0.8609)
Epoch 15 | loss=0.3015 | avg_dev_f1=0.7823 (opinion_label: 0.7037 | misinfo
rmation_label: 0.8609)
Epoch 16 | loss=0.2686 | avg_dev_f1=0.7798 (opinion_label: 0.6838 | misinfo
rmation_label: 0.8758)

```

Early stopping (no improvement for 5 epochs). Best F1: 0.7852

CNN MTL — POS & Linguistic Features (Sprint 3)

```

In [ ]: # Sprint 3 CNN MTL with POS tag features and annotated linguistic features a

import cnn_mtl_ling as cnn_mtl_ling

# need new data loaders to handle inclusion of pos and ling in batches
train_loader_ling = cnn_mtl_ling.make_loader(train_rows, vocab, shuffle=True)
dev_loader_ling = cnn_mtl_ling.make_loader(dev_rows, vocab, shuffle=False, t

for target in TARGETS:
    model = cnn_mtl_ling.TextCNN(len(vocab), embed_matrix).to(DEVICE)
    model = cnn_mtl_ling.train_model(model, train_loader_ling, dev_loader_li
        train_targets=[target])
    key = f"TextCNN MTL (POS + linguistic) - {target}"
    results[key] = cnn_mtl_ling.predict(model, dev_loader_ling, DEVICE, targ

#save metrics for use in final table
metrics_cache[key] = compute_metrics(*results[key])

```

Epoch	loss	avg_dev_f1	opinion_label
Epoch 1	0.7563	0.6922	0.6922
Epoch 2	0.6484	0.6795	0.6795
Epoch 3	0.5950	0.6869	0.6869
Epoch 4	0.5511	0.7138	0.7138
Epoch 5	0.4929	0.7138	0.7138
Epoch 6	0.4376	0.7097	0.7097
Epoch 7	0.3947	0.7064	0.7064
Epoch 8	0.3346	0.7042	0.7042
Epoch 9	0.2858	0.7172	0.7172
Epoch 10	0.2505	0.7118	0.7118
Epoch 11	0.2141	0.7042	0.7042
Epoch 12	0.1941	0.7017	0.7017
Epoch 13	0.1583	0.7050	0.7050
Epoch 14	0.1369	0.7089	0.7089

Early stopping (no improvement for 5 epochs). Best F1: 0.7172

Epoch	loss	avg_dev_f1	misinformation_label
Epoch 1	1.0009	0.7415	0.7415
Epoch 2	0.8683	0.7492	0.7492
Epoch 3	0.6996	0.7717	0.7717
Epoch 4	0.5201	0.7921	0.7921
Epoch 5	0.3832	0.8245	0.8245
Epoch 6	0.3005	0.8550	0.8550
Epoch 7	0.2377	0.8627	0.8627
Epoch 8	0.1772	0.8571	0.8571
Epoch 9	0.1293	0.8456	0.8456
Epoch 10	0.1003	0.8609	0.8609
Epoch 11	0.0810	0.8609	0.8609
Epoch 12	0.0706	0.8609	0.8609

Early stopping (no improvement for 5 epochs). Best F1: 0.8627

CNN MTL - POS Features only (Sprint 3)

In []: *#Sprint 3 CNN MTL with POS tag features only added*

```
import cnn_mtl_no_ling as cnn_mtl_pos

# need new data loaders to handle inclusion of pos in batches
train_loader_pos = cnn_mtl_pos.make_loader(train_rows, vocab, shuffle=True,
dev_loader_pos = cnn_mtl_pos.make_loader(dev_rows, vocab, shuffle=False, tok

for target in TARGETS:
    model = cnn_mtl_pos.TextCNN(len(vocab), embed_matrix).to(DEVICE)
    model = cnn_mtl_pos.train_model(model, train_loader_pos, dev_loader_pos,
                                     train_targets=[target])
    key = f"TextCNN MTL (POS) - {target}"
    results[key] = cnn_mtl_pos.predict(model, dev_loader_pos, DEVICE, target

#save metrics for use in final table
metrics_cache[key] = compute_metrics(*results[key])
```

Epoch	1	loss=0.7568	avg_dev_f1=0.6875 (opinion_label: 0.6875)
Epoch	2	loss=0.6512	avg_dev_f1=0.6600 (opinion_label: 0.6600)
Epoch	3	loss=0.6036	avg_dev_f1=0.7097 (opinion_label: 0.7097)
Epoch	4	loss=0.5496	avg_dev_f1=0.6897 (opinion_label: 0.6897)
Epoch	5	loss=0.4953	avg_dev_f1=0.7124 (opinion_label: 0.7124)
Epoch	6	loss=0.4382	avg_dev_f1=0.7177 (opinion_label: 0.7177)
Epoch	7	loss=0.3884	avg_dev_f1=0.7093 (opinion_label: 0.7093)
Epoch	8	loss=0.3375	avg_dev_f1=0.7152 (opinion_label: 0.7152)
Epoch	9	loss=0.2961	avg_dev_f1=0.7261 (opinion_label: 0.7261)
Epoch	10	loss=0.2445	avg_dev_f1=0.6945 (opinion_label: 0.6945)
Epoch	11	loss=0.2292	avg_dev_f1=0.7166 (opinion_label: 0.7166)
Epoch	12	loss=0.1811	avg_dev_f1=0.7261 (opinion_label: 0.7261)
Epoch	13	loss=0.1598	avg_dev_f1=0.7238 (opinion_label: 0.7238)
Epoch	14	loss=0.1420	avg_dev_f1=0.7362 (opinion_label: 0.7362)
Epoch	15	loss=0.1081	avg_dev_f1=0.7362 (opinion_label: 0.7362)
Epoch	16	loss=0.0922	avg_dev_f1=0.7326 (opinion_label: 0.7326)
Epoch	17	loss=0.0891	avg_dev_f1=0.7117 (opinion_label: 0.7117)
Epoch	18	loss=0.0767	avg_dev_f1=0.7106 (opinion_label: 0.7106)
Epoch	19	loss=0.0651	avg_dev_f1=0.7267 (opinion_label: 0.7267)

Early stopping (no improvement for 5 epochs). Best F1: 0.7362

Epoch	1	loss=1.0068	avg_dev_f1=0.6980 (misinformation_label: 0.6980)
Epoch	2	loss=0.8588	avg_dev_f1=0.8051 (misinformation_label: 0.8051)
Epoch	3	loss=0.7007	avg_dev_f1=0.7864 (misinformation_label: 0.7864)
Epoch	4	loss=0.5176	avg_dev_f1=0.8370 (misinformation_label: 0.8370)
Epoch	5	loss=0.3792	avg_dev_f1=0.8550 (misinformation_label: 0.8550)
Epoch	6	loss=0.3064	avg_dev_f1=0.8571 (misinformation_label: 0.8571)
Epoch	7	loss=0.2105	avg_dev_f1=0.8644 (misinformation_label: 0.8644)
Epoch	8	loss=0.1690	avg_dev_f1=0.8609 (misinformation_label: 0.8609)
Epoch	9	loss=0.1393	avg_dev_f1=0.8684 (misinformation_label: 0.8684)
Epoch	10	loss=0.0970	avg_dev_f1=0.8684 (misinformation_label: 0.8684)
Epoch	11	loss=0.0801	avg_dev_f1=0.8684 (misinformation_label: 0.8684)
Epoch	12	loss=0.0640	avg_dev_f1=0.8456 (misinformation_label: 0.8456)
Epoch	13	loss=0.0514	avg_dev_f1=0.8758 (misinformation_label: 0.8758)
Epoch	14	loss=0.0450	avg_dev_f1=0.8533 (misinformation_label: 0.8533)
Epoch	15	loss=0.0398	avg_dev_f1=0.8684 (misinformation_label: 0.8684)
Epoch	16	loss=0.0308	avg_dev_f1=0.8609 (misinformation_label: 0.8609)
Epoch	17	loss=0.0320	avg_dev_f1=0.8758 (misinformation_label: 0.8758)
Epoch	18	loss=0.0256	avg_dev_f1=0.8609 (misinformation_label: 0.8609)

Early stopping (no improvement for 5 epochs). Best F1: 0.8758

Logistic Regression Baseline

```
In [ ]: # Run Logistic Regression model – trained separately per target
import logreg_baseline as lr

for target in TARGETS:
    key = f"LogReg – {target}"
    results[key] = lr.run(train_rows, dev_rows, task=target)

    #save metrics for use in final table
    metrics_cache[key] = compute_metrics(*results[key])
```

```
— Logistic Regression [opinion_label] —  
Building features (TF-IDF + linguistic)  
Feature matrix: train=(600, 2504), dev=(200, 2504)  
Running GridSearchCV over C  
Best C: 1.0 | CV macro-F1: 0.7013
```

```
— Logistic Regression [misinformation_label] —  
Building features (TF-IDF + linguistic)  
Feature matrix: train=(600, 2504), dev=(200, 2504)  
Running GridSearchCV over C  
Best C: 1.0 | CV macro-F1: 0.8514
```

Logistic Regression Transfer Learning

```
In [ ]: # Run Logistic Regression transfer model – trained separately per target  
import logreg_transfer as lr_transfer  
  
for target in TARGETS:  
    key = f"LogReg (+ embeddings) – {target}"  
    results[key] = lr_transfer.run(train_rows, dev_rows, task=target)  
  
#save metrics for use in final table  
metrics_cache[key] = compute_metrics(*results[key])
```

```
— Logistic Regression [opinion_label] —  
Building features (TF-IDF, linguistic features, FastText embeddings)  
Loading FastText vectors from /Users/jennifer/.cache/huggingface/hub/dataset  
s--COLX523--fasttext-cc-en-300/snapshots/63ba06b23eb770a6fe0f6c17b0de981e43a  
ec6aa/cc.en.300.vec ...  
3566/4375 vocab tokens found in FastText vectors (81.5%)  
Feature matrix: train=(600, 2804), dev=(200, 2804)  
Running GridSearchCV over C  
Best C: 1.0 | CV macro-F1: 0.7285
```

```
— Logistic Regression [misinformation_label] —  
Building features (TF-IDF, linguistic features, FastText embeddings)  
Loading FastText vectors from /Users/jennifer/.cache/huggingface/hub/dataset  
s--COLX523--fasttext-cc-en-300/snapshots/63ba06b23eb770a6fe0f6c17b0de981e43a  
ec6aa/cc.en.300.vec ...  
3566/4375 vocab tokens found in FastText vectors (81.5%)  
Feature matrix: train=(600, 2804), dev=(200, 2804)  
Running GridSearchCV over C  
Best C: 1.0 | CV macro-F1: 0.8716
```

LogReg MTL — Cascaded POS Prediction (Sprint 3)

```
In [ ]: # Sprint 3 MTL: Logistic Regression with cascaded POS distribution features  
import logreg_mtl as lr_mtl  
  
for target in TARGETS:  
    key = f"LogReg MTL (cascaded POS) – {target}"  
    results[key] = lr_mtl.run(train_rows, dev_rows, task=target)  
  
#save metrics for use in final table  
metrics_cache[key] = compute_metrics(*results[key])
```



```

— LogReg MTL Cascaded POS [opinion_label] —
  Building features (TF-IDF, FastText, linguistic, cascaded POS)
Loading FastText vectors from /Users/jennifer/.cache/huggingface/hub/dataset
s---COLX523---fasttext-cc-en-300/snapshots/63ba06b23eb770a6fe0f6c17b0de981e43a
ec6aa/cc.en.300.vec ...
  3566/4375 vocab tokens found in FastText vectors (81.5%)
  Running secondary task: POS tagging 600 documents ...
  POS distribution feature dim: 15 tags
  Running secondary task: POS tagging 200 documents ...
  POS distribution feature dim: 15 tags
  Feature matrix: train=(600, 2819), dev=(200, 2819)
  Running GridSearchCV over C
  Best C: 0.1 | CV macro-F1: 0.7188

— LogReg MTL Cascaded POS [misinformation_label] —
  Building features (TF-IDF, FastText, linguistic, cascaded POS)
Loading FastText vectors from /Users/jennifer/.cache/huggingface/hub/dataset
s---COLX523---fasttext-cc-en-300/snapshots/63ba06b23eb770a6fe0f6c17b0de981e43a
ec6aa/cc.en.300.vec ...
  3566/4375 vocab tokens found in FastText vectors (81.5%)
  Running secondary task: POS tagging 600 documents ...
  POS distribution feature dim: 15 tags
  Running secondary task: POS tagging 200 documents ...
  POS distribution feature dim: 15 tags
  Feature matrix: train=(600, 2819), dev=(200, 2819)
  Running GridSearchCV over C
  Best C: 1.0 | CV macro-F1: 0.8461

```

Simple Ensembling

```

In [ ]: # Soft vote ensemble
import importlib
import simple_ensemble
importlib.reload(simple_ensemble)
from simple_ensemble import soft_vote

for task in TARGETS:
    preds, labels, probs = soft_vote([
        results[f"LogReg - {task}"],
        results[f"LogReg (+ embeddings) - {task}"],
        results[f"LogReg MTL (cascaded POS) - {task}"],
        results[f"TextCNN - {task}"],
        results[f"TextCNN Transfer - {task}"],
        results[f"TextCNN MTL (POS + linguistic) - {task}"],
        results[f"TextCNN MTL (POS) - {task}"],
    ])
    key = f"Soft Vote Ensemble - {task}"
    results[key] = (preds, labels, probs)
    #save metrics for use in final table
    metrics_cache[key] = compute_metrics(preds, labels, probs)

pd.set_option("display.float_format", "{:.4f}".format)
pd.DataFrame([{"Model": f"Soft Vote Ensemble - {t}", **{
    "Macro F1": metrics_cache[f"Soft Vote Ensemble - {t}"]["macro_f1"],
    "F1 (not-op)": metrics_cache[f"Soft Vote Ensemble - {t}"]["f1_class0"],

```



```

    "F1 (opinion)": metrics_cache[f"Soft Vote Ensemble - {t}"]["f1_class1"],
    "F1.5 (recall-weighted)": metrics_cache[f"Soft Vote Ensemble - {t}"]["f1_5"],
    "AUC-ROC": metrics_cache[f"Soft Vote Ensemble - {t}"]["auc_roc"],
  }) for t in TARGETS]).set_index("Model")

```

Out[]:

	Macro F1	F1 (not- op)	F1 (opinion)	F1.5 (recall- weighted)	AUC- ROC
Model					
Soft Vote Ensemble — opinion_label	0.7267	0.7568	0.6966	0.7152	0.7957
Soft Vote Ensemble — misinformation_label	0.9031	0.9492	0.8571	0.8603	0.9689

Motivated Ensembling

```

In [12]: # Motivated (F1-weighted) ensemble
import importlib
import motivated_ensemble
importlib.reload(motivated_ensemble)
from motivated_ensemble import motivated_soft_vote

model_order = [
    "LogReg",
    "LogReg (+ embeddings)",
    "LogReg MTL (cascaded POS)",
    "TextCNN",
    "TextCNN Transfer",
    "TextCNN MTL (POS + linguistic)",
    "TextCNN MTL (POS)",
]

for task in TARGETS:
    f1_weights = [metrics_cache[f"{m} - {task}"]["macro_f1"] for m in model_order]
    preds, labels, probs = motivated_soft_vote(
        model_outputs=[results[f"{m} - {task}"] for m in model_order],
        f1_weights=f1_weights,
    )
    key = f"Motivated Ensemble - {task}"
    results[key] = (preds, labels, probs)
    metrics_cache[key] = compute_metrics(preds, labels, probs)

pd.set_option("display.float_format", "{:.4f}".format)
pd.DataFrame([{"Model": f"Motivated Ensemble - {t}", **{
    "Macro F1": metrics_cache[f"Motivated Ensemble - {t}"]["macro_f1"],
    "F1 (not-op)": metrics_cache[f"Motivated Ensemble - {t}"]["f1_class0"],
    "F1 (opinion)": metrics_cache[f"Motivated Ensemble - {t}"]["f1_class1"],
    "F1.5 (recall-weighted)": metrics_cache[f"Motivated Ensemble - {t}"]["f1_5"],
    "AUC-ROC": metrics_cache[f"Motivated Ensemble - {t}"]["auc_roc"],
  }} for t in TARGETS]).set_index("Model")

```

Threshold sweep (Macro F1 criterion):

Threshold	Macro F1	F1 (cl.0)	F1 (cl.1)	Accuracy
0.30	0.6572	0.6264	0.6881	0.6600
0.35	0.7099	0.7041	0.7157	0.7100
0.40	0.7299	0.7353	0.7245	0.7300
0.45	0.7190	0.7358	0.7021	0.7200
0.50	0.7267	0.7568	0.6966	0.7300
0.55	0.7126	0.7586	0.6667	0.7200
0.60	0.6923	0.7661	0.6184	0.7100
0.65	0.6917	0.7765	0.6069	0.7150
0.70	0.6581	0.7778	0.5385	0.7000

Selected threshold: 0.40

Threshold sweep (Macro F1 criterion):

Threshold	Macro F1	F1 (cl.0)	F1 (cl.1)	Accuracy
0.30	0.8896	0.9371	0.8421	0.9100
0.35	0.9065	0.9481	0.8649	0.9250
0.40	0.9065	0.9481	0.8649	0.9250
0.45	0.9102	0.9524	0.8679	0.9300
0.50	0.9090	0.9527	0.8654	0.9300
0.55	0.8947	0.9463	0.8431	0.9200
0.60	0.8725	0.9369	0.8081	0.9050
0.65	0.8550	0.9311	0.7789	0.8950
0.70	0.8364	0.9256	0.7473	0.8850

Selected threshold: 0.45

Out[12]:

	Macro F1	F1 (not-op)	F1 (opinion)	F1.5 (recall-weighted)	AUC-ROC
Model					
Motivated Ensemble — opinion_label	0.7299	0.7353	0.7245	0.7698	0.7967
Motivated Ensemble — misinformation_label	0.9102	0.9524	0.8679	0.8743	0.9687

Lean Ensemble Soft Vote: uses best logreg and best CNN model only (Sprint 3)

```
In [13]: # Lean soft vote: best LogReg + best CNN per task (selected dynamically from

cnn_prefixes = ["TextCNN", "TextCNN Transfer", "TextCNN MTL (POS + linguistic
lr_prefixes = ["LogReg", "LogReg (+ embeddings)", "LogReg MTL (cascaded POS

best_cnn = {
    task: max(cnn_prefixes, key=lambda m: metrics_cache[f"{m} - {task}"]["ma
    for task in TARGETS
}
best_logreg = {
    task: max(lr_prefixes, key=lambda m: metrics_cache[f"{m} - {task}"]["mac
    for task in TARGETS
```

```

}

for task in TARGETS:
    print(f"{task}: best LogReg = {best_logreg[task]} | best CNN = {best_cnn[task]}")

for task in TARGETS:
    preds, labels, probs = soft_vote([
        results[f"{best_logreg[task]} - {task}"],
        results[f"{best_cnn[task]} - {task}"],
    ])
    key = f"Lean Soft Vote - {task}"
    results[key] = (preds, labels, probs)
    metrics_cache[key] = compute_metrics(preds, labels, probs)

pd.DataFrame([{"Model": f"Lean Soft Vote - {t}", **{
    "Macro F1": metrics_cache[f"Lean Soft Vote - {t}"]["macro_f1"],
    "F1 (not-op)": metrics_cache[f"Lean Soft Vote - {t}"]["f1_class0"],
    "F1 (opinion)": metrics_cache[f"Lean Soft Vote - {t}"]["f1_class1"],
    "F1.5 (recall-weighted)": metrics_cache[f"Lean Soft Vote - {t}"]["fbeta_15"],
    "AUC-ROC": metrics_cache[f"Lean Soft Vote - {t}"]["auc_roc"],
}} for t in TARGETS]).set_index("Model")

```

opinion_label: best LogReg = LogReg (+ embeddings) | best CNN = TextCNN MTL (POS)
 misinformation_label: best LogReg = LogReg (+ embeddings) | best CNN = TextCNN

Out[13]:

	Macro F1	F1 (not-op)	F1 (opinion)	F1.5 (recall-weighted)	AUC-ROC
Model					
Lean Soft Vote — opinion_label	0.7159	0.7500	0.6818	0.6971	0.8014
Lean Soft Vote — misinformation_label	0.9161	0.9559	0.8762	0.8794	0.9676

Lean Motivated Ensemble: uses best CNN and best logreg models only (Sprint 3)

In [14]: *# Lean motivated ensemble: same 2 models, F1-weighted with threshold sweep*

```

for task in TARGETS:
    lean_models = [best_logreg[task], best_cnn[task]]
    f1_weights = [metrics_cache[f"{m} - {task}"]["macro_f1"] for m in lean_models]
    preds, labels, probs = motivated_soft_vote(
        model_outputs=[results[f"{m} - {task}"] for m in lean_models],
        f1_weights=f1_weights,
    )
    key = f"Lean Motivated Ensemble - {task}"
    results[key] = (preds, labels, probs)
    metrics_cache[key] = compute_metrics(preds, labels, probs)

pd.DataFrame([{"Model": f"Lean Motivated Ensemble - {t}", **{
    "Macro F1": metrics_cache[f"Lean Motivated Ensemble - {t}"]["macro_f1"],

```

```

"F1 (not-op)": metrics_cache[f"Lean Motivated Ensemble - {t}"]["f1_class0"]
"F1 (opinion)": metrics_cache[f"Lean Motivated Ensemble - {t}"]["f1_class1"]
"F1.5 (recall-weighted)": metrics_cache[f"Lean Motivated Ensemble - {t}"]["f1.5"]
"AUC-ROC": metrics_cache[f"Lean Motivated Ensemble - {t}"]["auc_roc"],
}} for t in TARGETS]).set_index("Model")

```

Threshold sweep (Macro F1 criterion):

Threshold	Macro F1	F1 (cl.0)	F1 (cl.1)	Accuracy
0.30	0.6937	0.6738	0.7136	0.6950
0.35	0.7250	0.7264	0.7236	0.7250 ◀
0.40	0.7248	0.7317	0.7179	0.7250
0.45	0.7186	0.7383	0.6989	0.7200
0.50	0.7206	0.7556	0.6857	0.7250
0.55	0.7060	0.7574	0.6545	0.7150
0.60	0.7163	0.7787	0.6538	0.7300
0.65	0.6980	0.7795	0.6164	0.7200
0.70	0.6681	0.7710	0.5652	0.7000

Selected threshold: 0.35

Threshold sweep (Macro F1 criterion):

Threshold	Macro F1	F1 (cl.0)	F1 (cl.1)	Accuracy
0.30	0.8637	0.9176	0.8099	0.8850
0.35	0.9065	0.9481	0.8649	0.9250
0.40	0.9122	0.9517	0.8727	0.9300
0.45	0.8985	0.9452	0.8519	0.9200
0.50	0.9161	0.9559	0.8762	0.9350 ◀
0.55	0.8947	0.9463	0.8431	0.9200
0.60	0.8800	0.9400	0.8200	0.9100
0.65	0.8629	0.9342	0.7917	0.9000
0.70	0.8364	0.9256	0.7473	0.8850

Selected threshold: 0.50

Out[14]:

	Macro F1	F1 (not-op)	F1 (opinion)	F1.5 (recall-weighted)	AUC-ROC
Model					
Lean Motivated Ensemble — opinion_label	0.7250	0.7264	0.7236	0.7729	0.8011
Lean Motivated Ensemble — misinformation_label	0.9161	0.9559	0.8762	0.8794	0.9676

All Results by Target

```

In [18]: # All results by target – everything is now in metrics_cache
all_rows = [{"Model": name, "Macro F1": m["macro_f1"],
              "F1 (not-op)": m["f1_class0"], "F1 (opinion)": m["f1_class1"],
              "F1.5 (recall-weighted)": m["f1.5"],
              "AUC-ROC": m.get("auc_roc", float("nan"))}]
for name, m in metrics_cache.items()

```

```

all_df = pd.DataFrame(all_rows)

for task, label in [("Opinion", "opinion_label"), ("Misinformation", "misinf
mask = all_df["Model"].str.endswith(f"- {label}")
df = (all_df[mask]
      .copy()
      .assign(Model=lambda d: d["Model"].str.replace(f" - {label}", ""),
      .set_index("Model")
      .sort_values("Macro F1", ascending=False))
print(task, "Ordered by Macro F1 (Desc)")
print("-" * 60)
display(df)
print()

```

Opinion Ordered by Macro F1 (Desc)

	Macro F1	F1 (not- op)	F1 (opinion)	F1.5 (recall- weighted)	AUC- ROC
Model					
TextCNN MTL (POS)	0.7362	0.7679	0.7045	0.7203	0.7913
Motivated Ensemble	0.7299	0.7353	0.7245	0.7698	0.7967
Soft Vote Ensemble	0.7267	0.7568	0.6966	0.7152	0.7957
TextCNN	0.7261	0.7589	0.6932	0.7087	0.7930
Lean Motivated Ensemble	0.7250	0.7264	0.7236	0.7729	0.8011
TextCNN MTL (POS + linguistic)	0.7172	0.7455	0.6889	0.7101	0.7917
Lean Soft Vote	0.7159	0.7500	0.6818	0.6971	0.8014
LogReg (+ embeddings)	0.7023	0.7306	0.6740	0.6962	0.7698
TextCNN Transfer	0.6963	0.7297	0.6629	0.6806	0.7701
LogReg MTL (cascaded POS)	0.6963	0.7297	0.6629	0.6806	0.7486
LogReg	0.6931	0.7391	0.6471	0.6530	0.7514

Misinformation Ordered by Macro F1 (Desc)

Model	Macro F1	F1 (not-op)	F1 (opinion)	F1.5 (recall-weighted)	AUC-ROC
Lean Soft Vote	0.9161	0.9559	0.8762	0.8794	0.9676
Lean Motivated Ensemble	0.9161	0.9559	0.8762	0.8794	0.9676
Motivated Ensemble	0.9102	0.9524	0.8679	0.8743	0.9687
Soft Vote Ensemble	0.9031	0.9492	0.8571	0.8603	0.9689
LogReg (+ embeddings)	0.8889	0.9428	0.8350	0.8318	0.9501
LogReg MTL (cascaded POS)	0.8831	0.9392	0.8269	0.8269	0.9157
LogReg	0.8784	0.9404	0.8163	0.7975	0.9630
TextCNN	0.8773	0.9356	0.8190	0.8221	0.9528
TextCNN MTL (POS)	0.8758	0.9360	0.8155	0.8125	0.9608
TextCNN Transfer	0.8742	0.9365	0.8119	0.8027	0.9593
TextCNN MTL (POS + linguistic)	0.8627	0.9293	0.7961	0.7932	0.9489

Best Model Analysis: Confusion Matrix & Examples

```
In [16]: def print_quadrant_examples(dev_rows, preds, labels, n=3):
    """Print up to n examples from each confusion matrix quadrant."""
    quadrants = {
        "True Positives (predicted=1, actual=1)": [],
        "True Negatives (predicted=0, actual=0)": [],
        "False Positives (predicted=1, actual=0)": [],
        "False Negatives (predicted=0, actual=1)": [],
    }
    for row, p, l in zip(dev_rows, preds, labels):
        if p == 1 and l == 1: quadrants["True Positives (predicted=1, actual=1)"].append(row)
        elif p == 0 and l == 0: quadrants["True Negatives (predicted=0, actual=0)"].append(row)
        elif p == 1 and l == 0: quadrants["False Positives (predicted=1, actual=0)"].append(row)
        elif p == 0 and l == 1: quadrants["False Negatives (predicted=0, actual=1)"].append(row)

    for label, rows in quadrants.items():
        print(f"\n— {label} ({len(rows)} total, showing {min(n, len(rows))} examples)")
        for r in rows[:n]:
            print(f"  [{r['id']}] {r['text'][:140]!r}")

    for task, label in [("Opinion", "opinion_label"), ("Misinformation", "misinformation_label")]:
        # pick best individual model for this task from metrics_cache
        task_keys = [k for k in metrics_cache if k.endswith(f"- {label}")]
        best_key = max(task_keys, key=lambda k: metrics_cache[k]["macro_f1"])
        preds, labels_list, probs = results[best_key]
```

```
m = metrics_cache[best_key]

print(f"\n{'='*60}")
print(f"{task} – best model: {best_key.replace(f' – {label}', '')}")
print(f"Macro F1: {m['macro_f1']:.4f} | AUC-ROC: {m.get('auc_roc', float('nan'))}")
print(f"\n{'='*60}")
print_confusion_matrix(preds, labels_list)
print_quadrant_examples(dev_rows, preds, labels_list, n=8)
```


=====

Opinion – best model: TextCNN MTL (POS)

Macro F1: 0.7362 | AUC-ROC: 0.7913

=====

Confusion matrix (rows=true, cols=predicted):

	pred=0	pred=1
true=0 (not-op):	86	31
true=1 (opinion):	21	62

— True Positives (predicted=1, actual=1) (62 total, showing 8) —

[3] 'im praying for all of my friends down in the Caribbean who have no where else to go and are forced to ride through the hurricane. be strong '

[7] 'The aftermath of a hurricane is horrific. The heat/humidity is excruciating, no water/ice, no bathing, complete darkness, bugs, no warm food'

[10] "@john19071969 It's unwise to flood the food supply and environment with new plant varieties. Good science requires more prudence."

[73] 'Irony just died a thousand deaths! ??? http://t.co/dBU300bDxz'

[121] 'Freshman: I wish Hurricane Dorian would come our way.\n\nMe, a senior: we had Harvey two years ago.\n\nFish: but no school\n\nMe: we flooded for d'

[131] 'As you know that Covid 19 has spread in India, our immune system must be strong to defeat it. I have some #TipsToBoostImmunity. \nSuch as do '

[135] '.@exxonmobil just announced that it will invest \$50 billion in the U.S. over the next 5 years. What does that mean for Americans? #jobsjobsj'

[181] "aunt marge you're blown up ehks"

— True Negatives (predicted=0, actual=0) (86 total, showing 8) —

[8] "No food, no FEMA: Hurricane Michael's survivors are furious – The Daily Beast <https://apple.news/APp4E5UMtQT2ULJVMPM0ovw\xa0...>"

[30] 'Many states along the Mid-Atlantic and the East Coast have shelters open in response to Hurricane Sandy. Search for an open shelter by...'

[42] 'The U.S. bombed Hiroshima 70 years ago today. A look at how war has changed since: <http://t.co/UQnj6nk9y3> <http://t.co/QLnnMxzFqK>'

[51] 'Shelter & warm clothing especially needed in blizzard areas. Open your hearts. <https://twitter.com/PalestinaSummer/status/6891795168796344>'

[54] '.@FaizaPatelBCJ w/ crucial context for Trump claim that 300 refugees now subjects of counterterrorism investigations <https://t.co/KZM6vj2xYr>'

[63] 'No Food, No FEMA: Hurricane Michael's Survivors Are Furious #SmartNews <https://www.thedailybeast.com/no-food-no-fema-hurricane-michaels-surv>'

[67] '#Cars in the #News– Florida man parks Smart car in kitchen to save it from Hurricane Dorian <https://www.autoblog.com/2019/09/05/hurricane-do>'

[108] 'CRS teams standing ready to respond to Hurricane #Matthew in #Haiti. Likely assistance to include shelter, relief supplies, water. @CRSnews'

— False Positives (predicted=1, actual=0) (31 total, showing 8) —

[17] 'Birds in a blizzard. We put out extra sunflower seeds since their usual food sources just got,Ä¶ <https://www.instagram.com/p/BBS2eYWgTbj/-t>'

[25] "I'm getting used to seeing a layer of ash on my car each morning – and I'm in AB. We've only seen a red sun this week. The smoke is as far "

[55] "@jihettly @esd2000 good morning! Yes I'm in the middle of the blizzard. I have plenty of food so I'm happy! Lol. Stay warm."

[59] 'Re: Hurricane Matthew: All of the @ASUTennis Student-Athletes are on their way to safety. Thanks to @ASU_Housing for arranging a safe place!'

[81] "Don't Panik! #KelbyTomlinson to the rescue! <http://t.co/hujvgsFLUs>"

[174] 'God damn it!!! I electrocuted myself ??'

[194] '@SpenceBailey1 flood buddy. thanks :). We still gonna play champion

s and eat unhealthy food one evening?'

[209] 'OMG! Gas prices affected by the hurricane!! Up 3 cents last night, 5 cents more before tomorrow!!!... oh wait, I ride a bike... nevermind...'

— False Negatives (predicted=0, actual=1) (21 total, showing 8) —

[2] 'His argument is correct. Hurricane Rita killed over 100 people on the road, drowned them in their own cars. pic.twitter.com/kulwIZW2EI'

[113] 'Photo Falsely Claims Obama Served Food To Hurricane Harvey Victims <http://www.westernjournalism.com/photo-falsely-claims-obama-served-food-h>'

[133] 'Team Trump's efforts to spin Mulvaney's quid pro quo confession are failing.\nMick Mulvaney and Mike Pompeo were tasked w/ spinning, but they'

[193] 'No Food, No FEMA: Hurricane Michael's Survivors Are Furious\nMiles and miles of Florida are obliterated, and residents have been left to fend'

[306] 'A total of 44 people have been killed in California's devastating wildfire as authorities continue to find bodies in burnt-out cars and home'

[388] 'Intense Footage Of A Helicopter Fighting A Wildfire Feet Away From Cars In California <http://on.digg.com/2PTKlV6-t>'

[432] 'Unlikely hurricane hero takes over chaotic Texas storm shelter: <http://p://aol.it/2wiRz8I-t> via @AOL\nZachary Dearing, HERO!!!'

[441] 'Hurricane #dorian finally moved on from the Bahamas, and photos of near-total devastation are trickling onto the internet. \n\nPeople will need'

=====
Misinformation – best model: Lean Soft Vote

Macro F1: 0.9161 | AUC-ROC: 0.9676
=====

Confusion matrix (rows=true, cols=predicted):

	pred=0	pred=1
true=0 (not-op):	141	7
true=1 (opinion):	6	46

— True Positives (predicted=1, actual=1) (46 total, showing 8) —

[54] '@FaizaPatelBCJ w/ crucial context for Trump claim that 300 refugees now subjects of counterterrorism investigations <https://t.co/KZM6vj2xYr>'

[73] 'Irony just died a thousand deaths! ??? <http://t.co/dBU300bDxz>'

[81] 'Don't Panik! #KelbyTomlinson to the rescue! <http://t.co/hujvgsFLUs>'

[131] 'As you know that Covid 19 has spread in India, our immune system must be strong to defeat it. I have some #TipsToBoostImmunity. \nSuch as do '

[133] 'Team Trump's efforts to spin Mulvaney's quid pro quo confession are failing.\nMick Mulvaney and Mike Pompeo were tasked w/ spinning, but they'

[169] 'Paul Rudd Emile Hirsch David Gordon Green 'Prince Avalanche' Q&A | Filmmakers at Google <http://t.co/e4QonKzndZ> #entretenimento #Video"

[174] 'God damn it!!! I electrocuted myself ??'

[181] 'aunt marge you're blown up ehks"

— True Negatives (predicted=0, actual=0) (141 total, showing 8) —

[2] 'His argument is correct. Hurricane Rita killed over 100 people on the road, drowned them in their own cars. pic.twitter.com/kulwIZW2EI'

[3] 'im praying for all of my friends down in the Caribbean who have nowhere else to go and are forced to ride through the hurricane. be strong '

[7] 'The aftermath of a hurricane is horrific. The heat/humidity is excruciating, no water/ice, no bathing, complete darkness, bugs, no warm food'

[8] 'No food, no FEMA: Hurricane Michael's survivors are furious – The Daily Beast <https://apple.news/APp4E5UMtQT2ULJVMpM0ovw\xa0...>"

[10] '@john19071969 It's unwise to flood the food supply and environment with new plant varieties. Good science requires more prudence."

[17] 'Birds in a blizzard. We put out extra sunflower seeds since their usual food sources just got,Ä¶ https://www.instagram.com/p/BBS2eYWgTbj/-t'

[25] "I'm getting used to seeing a layer of ash on my car each morning – and I'm in AB. We've only seen a red sun this week. The smoke is as far "

[30] 'Many states along the Mid-Atlantic and the East Coast have shelters open in response to Hurricane Sandy. Search for an open shelter by...'

— False Positives (predicted=1, actual=0) (7 total, showing 7) —

[42] 'The U.S. bombed Hiroshima 70 years ago today. A look at how war has changed since: http://t.co/UQnj6nk9y3 http://t.co/QLnnMxzFqK'

[135] '@exxonmobil just announced that it will invest \$50 billion in the U.S. over the next 5 years. What does that mean for Americans? #jobsjobsj'

[242] '@TheSmallClark 'He'll kill me instead if he survived the shot. I don't exactly know. I fled the scene and pulled the trigger with---"

[323] 'Investigators say a fatal Virgin Galactic spaceship crash last year was caused by structural failure after the co\x89Û_'

[520] 'The proportion of patients spending over 4 hours in A&E has substantially increased in recent years. Are waiting lists, ambulance data a'

[723] 'RT AbbsWinston: #Zionist #Terrorist kidnapped 15 #Palestinians in overnight terror on Palestinian Villages \x89Û_ http://t.co/J5mKcbKcov'

[811] '#Coronavirus #Research:\nDroplets in air from sneeze or #coughing can #infect even after 3 hours. Once deposited on steel/plastic surface, #v'

— False Negatives (predicted=0, actual=1) (6 total, showing 6) —

[212] 'Fear and panic in the air I want to be free from desolation and despair!'

[540] '@StevenOnTwitter @PussyxDestroyer just order a blizzard pay then put your nuts in it say they have you ball flavored. Boom free ice cream'

[559] 'Kids are inundated with images and information online and in media and have no way to deconstruct. – Kerri Sackville #TMS7'

[565] '#MissionHills CA #Nursing : Registered Nurse – Emergency Department (Full Time... at Providence Health & Services http://t.co/Z5grLREy6'

[659] 'Weapons: Twin Knives #0jouBot'

[868] '@afterShock_DeLo scuf ps live and the game... cya'

Detailed Analysis

This notebook includes all of the previous models from Sprint 1 and Sprint 2 and introduces new models. The new models include:

- Logistic Regression with POS tagging for Multi-Task Learning
- CNN with POS tagging for Multi-Task Learning
- CNN with both linguistic marker features and POS tagging for Multi-Task Learning
- Lean Motivated Ensemble that uses only the best LogReg + best CNN models per target
- Lean Soft Vote Ensemble that uses only the best LogReg + best CNN models per target

We have been creating, modifying and comparing models for two different targets: opinion and misinformation. All of our models are based on CNN, Logistic Regression, or an ensemble with both types. One of the interesting results is that different models

perform better on different tasks. All Logistic Regression models outperform all CNN models on the misinformation classification task. However, all CNN models outperform all Logistic Regression models on the opinion classification task, with one exception. The exception is the TextCNN Transfer model, which was tied with LogReg MTL and surpassed by LogReg (+ embeddings). Overall, ensembles help classification in both tasks.

With a highest Macro F1 score at 0.7362, the opinion classification task appears to be the more difficult task. According to this metric, TextCNN MTL (POS) outperformed all of the other models. However, Lean Motivated Ensemble was only slightly behind it with a score of 0.7347. Notably, TextCNN MTL (POS + linguistic) did not perform as well as TextCNN MTL (POS). This suggests that our annotated linguistic markers weaken the learning signal for that model.

Examples of false positives for our best opinion classifier suggest that the model is potentially using pronouns as a signal for opinion. Most of the eight shown examples feature casual writing and use of pronouns. Examples, like [17] 'Birds in a blizzard. We put out extra sunflower seeds since their usual food sources just got...' are personal observations and should not be labeled as 'opinion.' Another example of features that may send a noisy signal is when all capital letters and exclamation points are used in factual statements. In the text, [209] 'OMG! Gas prices affected by the hurricane!! Up 3 cents last night, 5 cents more before tomorrow!!!... oh wait, I ride a bike... nevermind...', the model likely sees the exclamation marks, use of all capital letters, and the use of the pronoun "I" as a signal of "opinion." Superficial markers of emphasis may be features that correlate with opinion, but they may also be used in non-opinion writing as well.

Overall, in our best opinion classification model, there were 21 cases of false negatives out of a total of 200 examples. However, there are some cases in the examples printed in this notebook that cause us to question the use and distinction of the opinion label overall. In the following example, [306] "A total of 44 people have been killed in California's devastating wildfire as authorities continue to find bodies in burnt-out cars and homes," We may agree with the model and not the "opinion" label. Different annotators may have different thoughts on what constitutes opinion. Therefore, it is understandable why a model may disagree with the opinion label at times. Other examples of false negatives include adjectives that the model may not read as indicating opinion, but our annotators did. Our annotators marked [388] "Intense Footage Of A Helicopter Fighting A Wildfire Feet Away From Cars In California" as opinion, likely due to the use of "intense." However, the model classified it as not opinion.

In the misinformation classification task, Lean Soft Vote and Lean Motivated Ensemble performed best by combining the two best performing models: LogReg (+ embeddings) and TextCNN MTL (POS). Overall, the models have consistently performed well on the misinformation task. Progress was made by limiting the number of models in the ensembles to include only 2, rather than all models created to date. Using all models in

an ensemble may weaken the learning signal when weaker models are contributing to the ensemble. Lean Soft Vote and Lean Motivated Ensemble scored the exact same score: Macro F1 = 0.9161. These models performed so well that looking at examples of false positives and false negatives may yield limited insights. Two of the examples of false negatives suggest annotation errors, rather than model errors. A nursing job posting [565] and a quote with an attribution [559] are both classified as not misinformation by the model, but annotated as misinformation. On those two examples, we agree with the model's classification. LogReg had a Macro F1 of 0.8784 and TextCNN had a Macro F1 of 0.8773. Improving the Macro F1 to 0.9161 with either of the two lean ensembles that use both TextCNN MTL (POS) and LogReg (+ embeddings) represents improvement to our misinformation classification.

Since our best Macro F1 score for misinformation classification is 0.9161 and our best for opinion classification is 0.7362, there is a significant gap in the models' ability to classify misinformation and opinion. Beyond the models themselves, there may be corpus-related reasons for this gap. Perhaps the task of annotating opinion is inherently more difficult or subjective and, therefore, more likely to have different annotations for the same text. Perhaps the text in our corpus is particularly difficult to annotate for opinion. Hopefully, all of the models will improve with more training data.